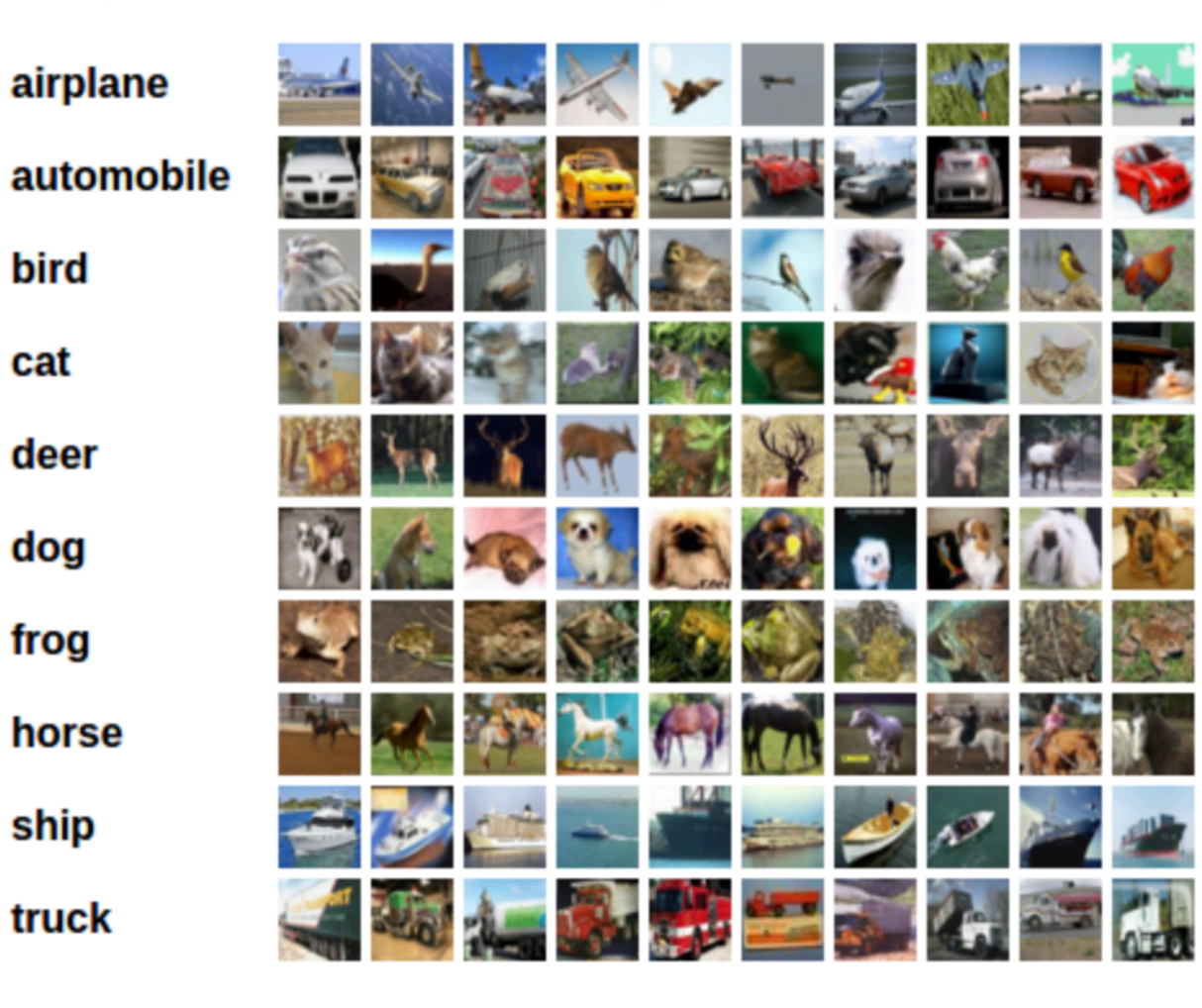
# Convolutional Neural Networks

In this notebook, we train a **CNN** to classify images from the CIFAR-10 database.

The images in this database are small color images that fall into one of ten classes; some example images are pictured below.



### Test for CUDA

Since these are larger (32x32x3) images, it may prove useful to speed up your training time by using a GPU. CUDA is a parallel computing platform and CUDA Tensors are the same as typical Tensors, only they utilize GPU's for computation.

In [2]:
```
# Import libraries
import torch
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

# PyTorch dataset
from torchvision import datasets
import torchvision.transforms as transforms
from torch.utils.data.sampler import SubsetRandomSampler

# PyTorch model
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
```

In [3]:
```
# check if CUDA is available
train_on_gpu = torch.cuda.is_available()

if not train_on_gpu:
    print('CUDA is not available.  Training on CPU ...')
else:
    print('CUDA is available!  Training on GPU ...')
```

```
CUDA is available!  Training on GPU ...
```

---

## Load the Data

Downloading may take a minute. We load in the training and test data, split the training data into a training and validation set, then create DataLoaders for each of these sets of data.

In [4]:
```
# number of subprocesses to use for data loading
num_workers = 0
# how many samples per batch to load
batch_size = 20
# percentage of training set to use as validation
valid_size = 0.2
```

In [5]:
```
# Data transform to convert data to a tensor and apply normalization

# augment train and validation dataset with RandomHorizontalFlip and RandomRotation
train_transform = transforms.Compose([
    transforms.RandomHorizontalFlip(), # randomly flip and rotate
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
    ])

test_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
    ])
```

In [6]:
```
# choose the training and test datasets
train_data = datasets.CIFAR10('data', train=True,
                              download=True, transform=train_transform)
test_data = datasets.CIFAR10('data', train=False,
                             download=True, transform=test_transform)
```

```
100%|██████████| 170M/170M [05:23<00:00, 528kB/s]
```

In [7]:
```
# obtain training indices that will be used for validation
num_train = len(train_data)
indices = list(range(num_train))
np.random.shuffle(indices)
split = int(np.floor(valid_size * num_train))
train_idx, valid_idx = indices[split:], indices[:split]
```

In [8]:
```
# define samplers for obtaining training and validation batches
train_sampler = SubsetRandomSampler(train_idx)
valid_sampler = SubsetRandomSampler(valid_idx)
```

In [9]:
```
# prepare data loaders (combine dataset and sampler)
train_loader = torch.utils.data.DataLoader(train_data, batch_size=batch_size,
    sampler=train_sampler, num_workers=num_workers)
valid_loader = torch.utils.data.DataLoader(train_data, batch_size=batch_size,
    sampler=valid_sampler, num_workers=num_workers)
test_loader = torch.utils.data.DataLoader(test_data, batch_size=batch_size,
    num_workers=num_workers)
```

In [10]:
```
# specify the image classes
classes = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
classes
```

Out[10]:
```
['airplane',
 'automobile',
 'bird',
 'cat',
 'deer',
 'dog',
 'frog',
 'horse',
 'ship',
 'truck']
```

### Visualize a Batch of Training Data

In [11]:
```
# helper function to un-normalize and display an image
def imshow(img):
    img = img / 2 + 0.5  # unnormalize
    plt.imshow(np.transpose(img, (1, 2, 0)))  # convert from Tensor image
```

In [13]:
```
# obtain one batch of training images
dataiter = iter(train_loader)
images, labels = next(dataiter)
images = images.numpy() # convert images to numpy for display
images.shape
```

Out[13]:
```
(20, 3, 32, 32)
```

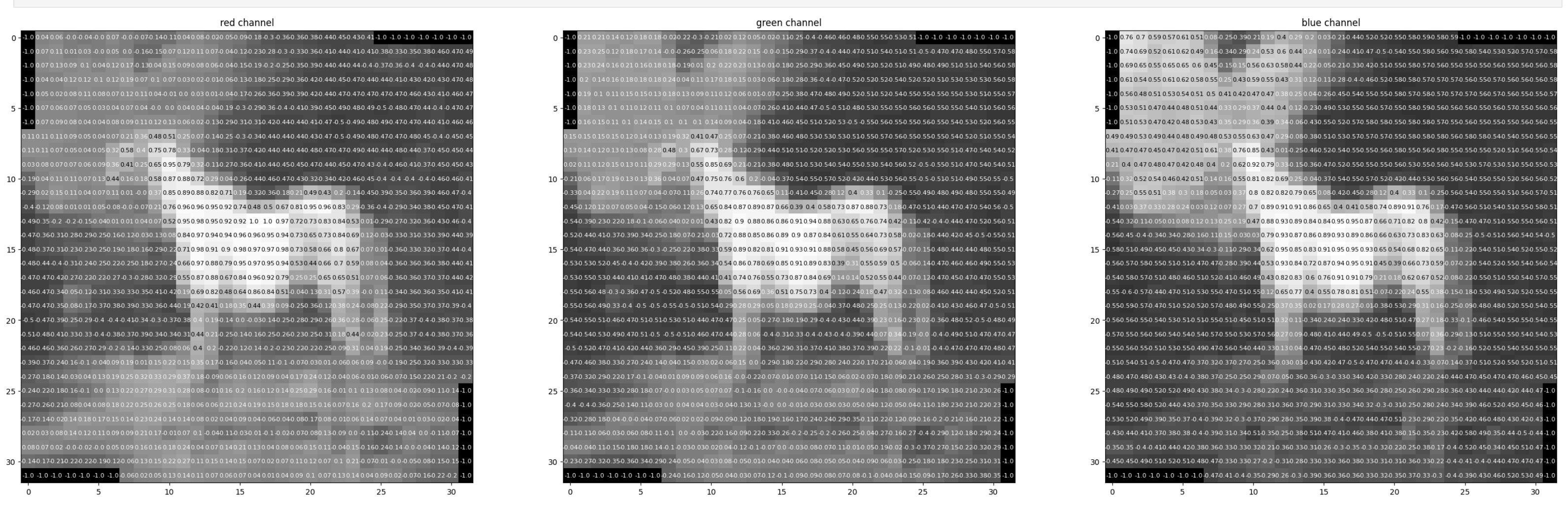
In [15]:
```
# plot the images in the batch, along with the corresponding labels
fig = plt.figure(figsize=(25, 4))
# display 20 images
for idx in np.arange(20):
    ax = fig.add_subplot(2, 20//2, idx+1, xticks=[], yticks=[])
    imshow(images[idx])
    ax.set_title(classes[labels[idx]])
```

### View an Image in More Detail

Here, we look at the normalized red, green, and blue (RGB) color channels as three separate, grayscale intensity images.

In [16]:
```
rgb_img = np.squeeze(images[3])
channels = ['red channel', 'green channel', 'blue channel']

fig = plt.figure(figsize = (36, 36))
for idx in np.arange(rgb_img.shape[0]):
    ax = fig.add_subplot(1, 3, idx + 1)
    img = rgb_img[idx]
    ax.imshow(img, cmap='gray')
    ax.set_title(channels[idx])
    width, height = img.shape
    thresh = img.max()/2.5
    for x in range(width):
        for y in range(height):
            val = round(img[x][y],2) if img[x][y] !=0 else 0
            ax.annotate(str(val), xy=(y,x),
                    horizontalalignment='center',
                    verticalalignment='center', size=8,
                    color='white' if img[x][y]<thresh else 'black')
```



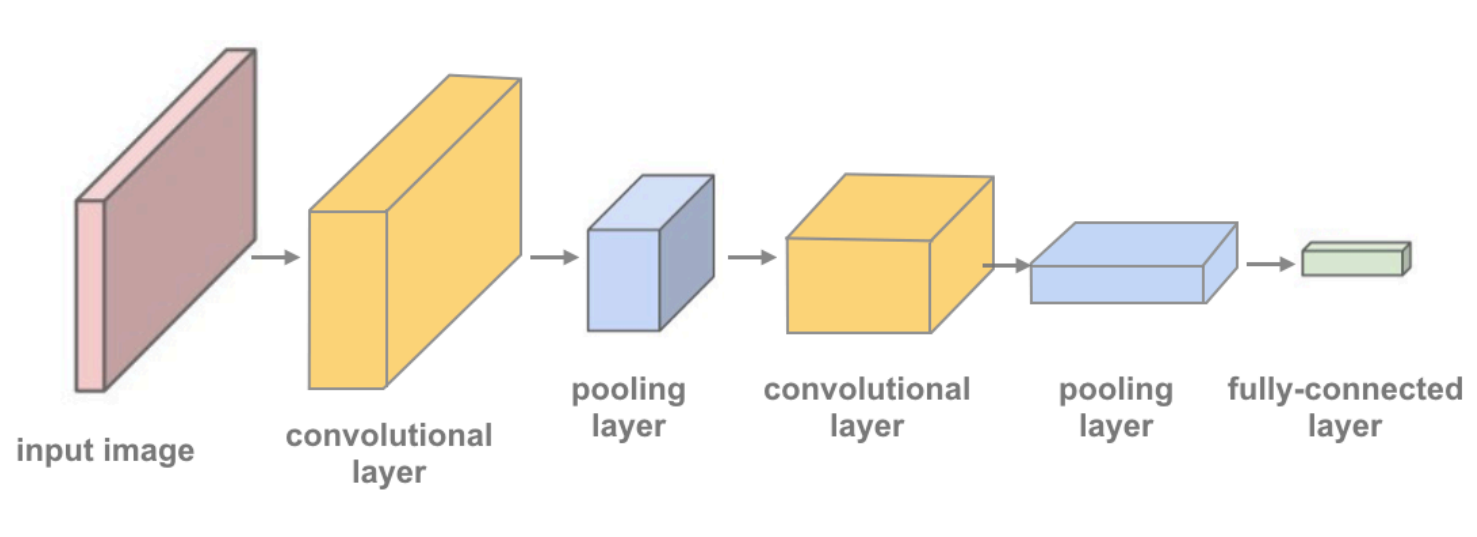
---

## Define the Network Architecture

This time, you'll define a CNN architecture. Instead of an MLP, which used linear, fully-connected layers, you'll use the following:

- Convolutional layers, which can be thought of as stack of filtered images.
- Maxpooling layers, which reduce the x-y size of an input, keeping only the most *active* pixels from the previous layer.
- The usual Linear + Dropout layers to avoid overfitting and produce a 10-dim output.

A network with 2 convolutional layers is shown in the image below and in the code, and you've been given starter code with one convolutional and one maxpooling layer.



### TODO: Define a model with multiple convolutional layers, and define the feedforward metwork behavior.

The more convolutional layers you include, the more complex patterns in color and shape a model can detect. It's suggested that your final model include 2 or 3 convolutional layers as well as linear layers + dropout in between to avoid overfitting.

It's good practice to look at existing research and implementations of related models as a starting point for defining your own models. You may find it useful to look at this PyTorch classification example or this Keras example to help decide on a final structure.

### Output volume for a convolutional layer

To compute the output size of a given convolutional layer we can perform the following calculation (taken from Stanford's cs231n course):

> We can compute the spatial size of the output volume as a function of the input volume size (W), the kernel/filter size (F), the stride with which they are applied (S), and the amount of zero padding used (P) on the border. The correct formula for calculating how many neurons define the output_W is given by `(W-F+2P)/S+1`.

For example for a 7x7 input and a 3x3 filter with stride 1 and pad 0 we would get a 5x5 output. With stride 2 we would get a 3x3 output.

In [17]:
```
# define the CNN architecture
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        # convolutional layer (sees 32x32x3 image tensor)
        self.conv1 = nn.Conv2d(3, 16, 3, padding=1)
        # convolutional layer (sees 16x16x16 tensor)
        self.conv2 = nn.Conv2d(16, 32, 3, padding=1)
        # convolutional layer (sees 8x8x32 tensor)
        self.conv3 = nn.Conv2d(32, 64, 3, padding=1)
        # max pooling layer
        self.pool = nn.MaxPool2d(2, 2)
        # linear layer (64 * 4 * 4 -> 500)
        self.fc1 = nn.Linear(64 * 4 * 4, 500)
        # linear layer (500 -> 10)
        self.fc2 = nn.Linear(500, 10)
        # dropout layer (p=0.25)
        self.dropout = nn.Dropout(0.25)

    def forward(self, x):
        # add sequence of convolutional and max pooling layers
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
        # flatten image input
        x = x.view(-1, 64 * 4 * 4)
        # add dropout layer
        x = self.dropout(x)
        # add 1st hidden layer, with relu activation function
        x = F.relu(self.fc1(x))
        # add dropout layer
        x = self.dropout(x)
        # add 2nd hidden layer, with relu activation function
        x = self.fc2(x)
        return x
```

In [18]:
```
# create a complete CNN
model = Net()
model
```

Out[18]:
```
Net(
  (conv1): Conv2d(3, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv2): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv3): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=1024, out_features=500, bias=True)
  (fc2): Linear(in_features=500, out_features=10, bias=True)
  (dropout): Dropout(p=0.25, inplace=False)
)
```

In [19]:
```
# move tensors to GPU if CUDA is available
if train_on_gpu:
    model.cuda()
```

### Specify Loss Function and Optimizer

Decide on a loss and optimization function that is best suited for this classification task. The linked code examples from above, may be a good starting point; this PyTorch classification example or this, more complex Keras example. Pay close attention to the value for **learning rate** as this value determines how your model converges to a small error.

### TODO: Define the loss and optimizer and see how these choices change the loss over time.

In [20]:
```
# specify loss function (categorical cross-entropy)
criterion = nn.CrossEntropyLoss()

# specify optimizer
optimizer = optim.SGD(model.parameters(), lr=0.01)
```

---

## Train the Network

Remember to look at how the training and validation loss decreases over time; if the validation loss ever increases it indicates possible overfitting. (In fact, in the below example, we could have stopped around epoch 33 or so!)

In [21]:
```
# number of epochs to train the model
n_epochs = 30

valid_loss_min = np.inf # track change in validation loss

for epoch in range(1, n_epochs+1):

    # keep track of training and validation loss
    train_loss = 0.0
    valid_loss = 0.0

    ###################
    # train the model #
    ###################
    model.train()
    for data, target in train_loader:
        # move tensors to GPU if CUDA is available
        if train_on_gpu:
            data, target = data.cuda(), target.cuda()
        # clear the gradients of all optimized variables
        optimizer.zero_grad()
        # forward pass: compute predicted outputs by passing inputs to the model
        output = model(data)
        # calculate the batch loss
        loss = criterion(output, target)
        # backward pass: compute gradient of the loss with respect to model parameters
        loss.backward()
        # perform a single optimization step (parameter update)
        optimizer.step()
        # update training loss
        train_loss += loss.item()*data.size(0)

    ######################
    # validate the model #
    ######################
    model.eval()
    for data, target in valid_loader:
        # move tensors to GPU if CUDA is available
        if train_on_gpu:
            data, target = data.cuda(), target.cuda()
        # forward pass: compute predicted outputs by passing inputs to the model
        output = model(data)
        # calculate the batch loss
        loss = criterion(output, target)
        # update average validation loss
        valid_loss += loss.item()*data.size(0)

    # calculate average losses
    train_loss = train_loss/len(train_loader.sampler)
    valid_loss = valid_loss/len(valid_loader.sampler)

    # print training/validation statistics
    print('Epoch: {} \tTraining Loss: {:.6f} \tValidation Loss: {:.6f}'.format(
        epoch, train_loss, valid_loss))

    # save model if validation loss has decreased
    if valid_loss <= valid_loss_min:
        print('Validation loss decreased ({:.6f} --> {:.6f}).  Saving model ...'.format(
        valid_loss_min,
        valid_loss))
        torch.save(model.state_dict(), 'model_cifar.pt')
        valid_loss_min = valid_loss
```

```
Epoch: 1 	Training Loss: 2.107825 	Validation Loss: 1.835478
Validation loss decreased (inf --> 1.835478).  Saving model ...
Epoch: 2 	Training Loss: 1.712487 	Validation Loss: 1.559635
Validation loss decreased (1.835478 --> 1.559635).  Saving model ...
Epoch: 3 	Training Loss: 1.524549 	Validation Loss: 1.418879
Validation loss decreased (1.559635 --> 1.418879).  Saving model ...
Epoch: 4 	Training Loss: 1.421445 	Validation Loss: 1.332058
Validation loss decreased (1.418879 --> 1.332058).  Saving model ...
Epoch: 5 	Training Loss: 1.342742 	Validation Loss: 1.251176
Validation loss decreased (1.332058 --> 1.251176).  Saving model ...
Epoch: 6 	Training Loss: 1.273103 	Validation Loss: 1.175724
Validation loss decreased (1.251176 --> 1.175724).  Saving model ...
Epoch: 7 	Training Loss: 1.209486 	Validation Loss: 1.144134
Validation loss decreased (1.175724 --> 1.144134).  Saving model ...
Epoch: 8 	Training Loss: 1.156574 	Validation Loss: 1.084330
Validation loss decreased (1.144134 --> 1.084330).  Saving model ...
Epoch: 9 	Training Loss: 1.110330 	Validation Loss: 1.024942
Validation loss decreased (1.084330 --> 1.024942).  Saving model ...
Epoch: 10 	Training Loss: 1.070402 	Validation Loss: 1.000482
Validation loss decreased (1.024942 --> 1.000482).  Saving model ...
Epoch: 11 	Training Loss: 1.031532 	Validation Loss: 0.952928
Validation loss decreased (1.000482 --> 0.952928).  Saving model ...
Epoch: 12 	Training Loss: 0.997355 	Validation Loss: 0.932777
Validation loss decreased (0.952928 --> 0.932777).  Saving model ...
Epoch: 13 	Training Loss: 0.971466 	Validation Loss: 0.917658
Validation loss decreased (0.932777 --> 0.917658).  Saving model ...
Epoch: 14 	Training Loss: 0.939129 	Validation Loss: 0.893451
Validation loss decreased (0.917658 --> 0.893451).  Saving model ...
Epoch: 15 	Training Loss: 0.915661 	Validation Loss: 0.886686
Validation loss decreased (0.893451 --> 0.886686).  Saving model ...
Epoch: 16 	Training Loss: 0.892441 	Validation Loss: 0.844551
Validation loss decreased (0.886686 --> 0.844551).  Saving model ...
Epoch: 17 	Training Loss: 0.863900 	Validation Loss: 0.814621
Validation loss decreased (0.844551 --> 0.814621).  Saving model ...
Epoch: 18 	Training Loss: 0.847035 	Validation Loss: 0.819158
Epoch: 19 	Training Loss: 0.825365 	Validation Loss: 0.823251
Epoch: 20 	Training Loss: 0.811396 	Validation Loss: 0.787719
Validation loss decreased (0.814621 --> 0.787719).  Saving model ...
Epoch: 21 	Training Loss: 0.789086 	Validation Loss: 0.783102
Validation loss decreased (0.787719 --> 0.783102).  Saving model ...
Epoch: 22 	Training Loss: 0.777538 	Validation Loss: 0.774646
Validation loss decreased (0.783102 --> 0.774646).  Saving model ...
Epoch: 23 	Training Loss: 0.762703 	Validation Loss: 0.743931
Validation loss decreased (0.774646 --> 0.743931).  Saving model ...
Epoch: 24 	Training Loss: 0.748316 	Validation Loss: 0.753226
Epoch: 25 	Training Loss: 0.740137 	Validation Loss: 0.729163
Validation loss decreased (0.743931 --> 0.729163).  Saving model ...
Epoch: 26 	Training Loss: 0.727256 	Validation Loss: 0.718606
Validation loss decreased (0.729163 --> 0.718606).  Saving model ...
Epoch: 27 	Training Loss: 0.705441 	Validation Loss: 0.726207
Epoch: 28 	Training Loss: 0.700978 	Validation Loss: 0.711362
Validation loss decreased (0.718606 --> 0.711362).  Saving model ...
Epoch: 29 	Training Loss: 0.684666 	Validation Loss: 0.694971
Validation loss decreased (0.711362 --> 0.694971).  Saving model ...
Epoch: 30 	Training Loss: 0.677006 	Validation Loss: 0.730932
```

### Load the Model with the Lowest Validation Loss

In [22]:
```
model.load_state_dict(torch.load('model_cifar.pt'))
```

Out[22]:
```
<All keys matched successfully>
```

---

## Test the Trained Network

Test your trained model on previously unseen data! A "good" result will be a CNN that gets around 70% (or more, try your best!) accuracy on these test images.

In [23]:
```
# track test loss
test_loss = 0.0
class_correct = list(0. for i in range(10))
class_total = list(0. for i in range(10))

model.eval()
# iterate over test data
for data, target in test_loader:
    # move tensors to GPU if CUDA is available
    if train_on_gpu:
        data, target = data.cuda(), target.cuda()
    # forward pass: compute predicted outputs by passing inputs to the model
    output = model(data)
    # calculate the batch loss
    loss = criterion(output, target)
    # update test loss
    test_loss += loss.item()*data.size(0)
    # convert output probabilities to predicted class
    _, pred = torch.max(output, 1)
    # compare predictions to true label
    correct_tensor = pred.eq(target.data.view_as(pred))
    correct = np.squeeze(correct_tensor.numpy()) if not train_on_gpu else np.squeeze(correct_tensor.cpu().numpy())
    # calculate test accuracy for each object class
    for i in range(batch_size):
        label = target.data[i]
        class_correct[label] += correct[i].item()
        class_total[label] += 1

# average test loss
test_loss = test_loss/len(test_loader.dataset)
print('Test Loss: {:.6f}\n'.format(test_loss))

for i in range(10):
    if class_total[i] > 0:
        print('Test Accuracy of %5s: %2d%% (%2d/%2d)' % (
            classes[i], 100 * class_correct[i] / class_total[i],
            np.sum(class_correct[i]), np.sum(class_total[i])))
    else:
        print('Test Accuracy of %5s: N/A (no training examples)' % (classes[i]))

print('\nTest Accuracy (Overall): %2d%% (%2d/%2d)' % (
    100. * np.sum(class_correct) / np.sum(class_total),
    np.sum(class_correct), np.sum(class_total)))
```

```
Test Loss: 0.662950

Test Accuracy of airplane: 84% (843/1000)
Test Accuracy of automobile: 88% (887/1000)
Test Accuracy of  bird: 67% (676/1000)
Test Accuracy of   cat: 61% (615/1000)
Test Accuracy of  deer: 66% (669/1000)
Test Accuracy of   dog: 65% (655/1000)
Test Accuracy of  frog: 85% (859/1000)
Test Accuracy of horse: 83% (836/1000)
Test Accuracy of  ship: 86% (868/1000)
Test Accuracy of truck: 79% (798/1000)

Test Accuracy (Overall): 77% (7706/10000)
```

### Question: What are your model's weaknesses and how might they be improved?

**Answer**: This model seems to do best on vehicles rather than animals. For example, it does best on the automobile class and worst on the cat class. I suspect it's because animals vary in color and size and so it would improve this model if I could increase the number of animal images in the first place or perhaps if I added another convolutional layer to detect finer patterns in these images. I could also experiment with a smaller learning rate so that the model takes small steps in the right direction as it is training.

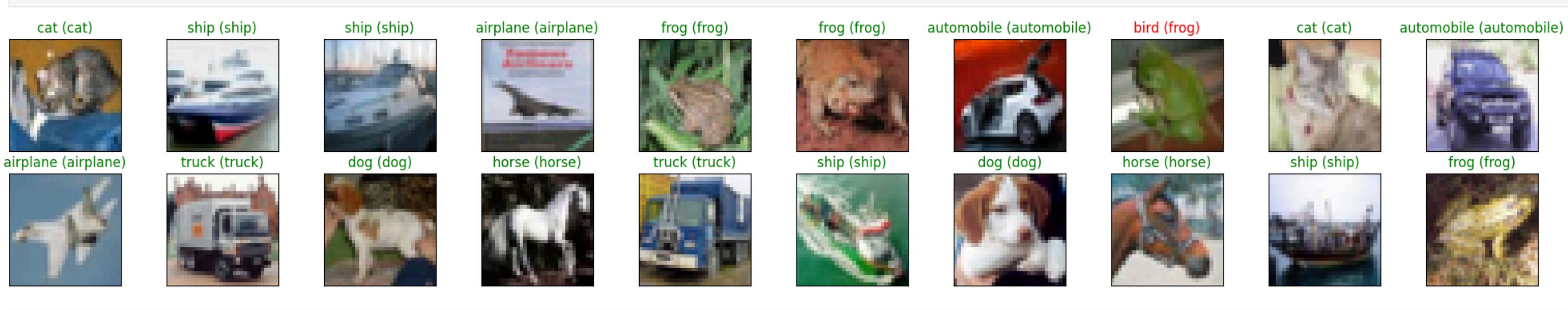
### Visualize Sample Test Results

In [26]:
```
# obtain one batch of test images
dataiter = iter(test_loader)
images, labels = next(dataiter)
images.numpy()

# move model inputs to cuda, if GPU available
if train_on_gpu:
    images = images.cuda()

# get sample outputs
output = model(images)
# convert output probabilities to predicted class
_, preds_tensor = torch.max(output, 1)
preds = np.squeeze(preds_tensor.numpy()) if not train_on_gpu else np.squeeze(preds_tensor.cpu().numpy())

# plot the images in the batch, along with predicted and true labels
fig = plt.figure(figsize=(25, 4))
for idx in np.arange(20):
    ax = fig.add_subplot(2, 20//2, idx+1, xticks=[], yticks=[])
    imshow(images[idx] if not train_on_gpu else images[idx].cpu())
    ax.set_title("{} ({})".format(classes[preds[idx]], classes[labels[idx]]),
                 color=("green" if preds[idx]==labels[idx].item() else "red"))
```



In [ ]: